

File Handling

Till now, we were taking the input from the console and writing it back to the console to interact with the user. Sometimes, it is not enough to only display the data on the console. The data to be displayed may be very large, and only a limited amount of data can be displayed on the console since the memory is volatile, it is impossible to recover the programmatically generated data again and again.

The file handling plays an important role when the data needs to be stored permanently into the file. A file is a named location on disk to store related information. We can access the stored information (non-volatile) after the program termination. The file-handling implementation is slightly lengthy or complicated in the other programming language, but it is easier and shorter in Python.

In Python, files are treated in two modes as text or binary. The file may be in the text or binary format, and each line of a file is ended with the special character.

Hence, a file operation can be done in the following order.

1. Open a file
2. Read or write - Performing operation
3. Close the file

Opening a file

Python provides an `open()` function that accepts two arguments, file name and access mode in which the file is accessed. The function returns a file object which can be used to perform various operations like reading, writing, etc.

Syntax:

```
file_object = open(<file-name>, <access-mode>)
```

The files can be accessed using various modes like read, write, or append. The following are the details about the access mode to open a file.

- r-*** It opens the file to read-only mode. The file pointer exists at the beginning. The file is by default open in this mode if no access mode is passed.
- rb-*** It opens the file to read-only in binary format. The file pointer exists at the beginning of the file.
- r+*** -It opens the file to read and write both. The file pointer exists at the beginning of the file.
- rb+*** -It opens the file to read and write both in binary format. The file pointer exists at the beginning of the file.
- w*** -It opens the file to write only. It overwrites the file if previously exists or creates a new one if no file exists with the same name. The file pointer exists at the beginning of the file.

- wb** -It opens the file to write only in binary format. It overwrites the file if it exists previously or creates a new one if no file exists. The file pointer exists at the beginning of the file.
- w+** -It opens the file to write and read both. It is different from r+ in the sense that it overwrites the previous file if one exists whereas r+ doesn't overwrite the previously written file. It creates a new file if no file exists. The file pointer exists at the beginning of the file.
- wb+** -It opens the file to write and read both in binary format. The file pointer exists at the beginning of the file.
- a** -It opens the file in the append mode. The file pointer exists at the end of the previously written file if exists any. It creates a new file if no file exists with the same name.
- ab** -It opens the file in the append mode in binary format. The pointer exists at the end of the previously written file. It creates a new file in binary format if no file exists with the same name.
- a+** -It opens a file to append and read both. The file pointer remains at the end of the file if a file exists. It creates a new file if no file exists with the same name.
- ab+** -It opens a file to append and read both in binary format. The file pointer remains at the end of the file.

The open() method

Let's look at the simple example to open a file named "sample.txt" (stored in the same directory or other) in read mode and printing its content on the console.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello World

```
#opens the file sample.txt in read mode
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r")
#open("C:\\Users\\Username\\Downloads\\sample.txt", "r")
#open("C:/Users/Username/Downloads/sample.txt", "r")
if fileptr:
    print("file is opened successfully")

#opens the file sample.txt located at same folder in read
mode
fileptr = open(r"sample.txt", "r")
if fileptr:
    print("file is opened successfully")
```

The close() method

Once all the operations are done on the file, we must close it through our Python script using the close() method. Any unwritten information gets destroyed once the close() method is called on a file object. We can perform any operation on the file externally using the file system which is the currently

opened in Python hence it is good practice to close the file once all the operations are done.

The syntax to use the close() method is given below.

Syntax

```
fileobject.close()
```

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello World

```
#opens and close the file sample.txt in read mode
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r")
if fileptr:
    print("File is opened successfully")
#closes the opened file
fileptr.close()
```

output:

File is opened successfully

After closing the file, we cannot perform any operation in the file. The file needs to be properly closed. If any exception occurs while performing some operations in the file then the program terminates without closing the file.

The with statement

The with statement was introduced in python 2.5. The with statement is useful in the case of manipulating the files. It is used in the scenario where a pair of statements is to be executed with a block of code in between.

The syntax to open a file using with the statement is given below.

```
with open(<file name>, <access mode>) as <file-pointer>:
```

```
    #statement suite
```

The advantage of using with statement is that it provides the guarantee to close the file regardless of how the nested block exits.

It is always suggestible to use the with statement in the case of files because, if the break, return, or exception occurs in the nested block of code then it automatically closes the file, we don't need to write the close() function. It doesn't let the file to corrupt.

```
with open(r"C:\Users\Username\Desktop\sample.txt", "r") as f:
    content = f.read()
    print(content)
```

Writing the file

To write some text to a file, we need to open the file using the open method with one of the following access modes.

- w:** It will overwrite the file if any file exists. The file pointer is at the beginning of the file.
- a:** It will append the existing file. The file pointer is at the end of the file. It creates a new file if no file exists.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello World

```
# open the sample.txt in write mode. Create a new file if no
such file exists.
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "w")
# overwriting the content to the file
fileptr.write(''Welcome to Python'') # file is modified to
Welcome to Python
# closing the opened the file
fileptr.close()
```

We have opened the file in w mode. The sample.txt file doesn't exist, it created a new file and we have written the content in the file using the write() function.

Create a sample.txt file in “C:\Users\Username\Desktop\sample.txt” and add following lines

Hello World

```
#open the sample.txt in append mode.
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "a")
#appending the content of the file
fileptr.write("Welcome to Python") #existing contents
appended with new content ie Welcome to Python
#closing the opened file
fileptr.close()
```

We can see that the content of the file is modified. We have opened the file in a mode and it appended the content in the existing sample.txt.

Reading the file

To read a file using the Python script, the Python provides the read() method. The read() method reads a string from the file. It can read the data in the text as well as a binary format.

The syntax of the read() method is given below.

Syntax:

`fileobj.read(<count>)`

Here, the count is the number of bytes to be read from the file starting from the beginning of the file. If the count is not specified, then it may read the content of the file until the end.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello World

```
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r")
#stores all the data of the file into the variable content
content = fileptr.read(5) #reads 5 bytes ie each character as a byte
#prints the content of the file
print(content)
#closes the opened file
fileptr.close()
```

output:

Hello

In the above code, we have read the content of sample.txt by using the read() function. We have passed count value as 5 which means it will read the first five characters from the file. If we use the following line, then it will print all content of the file.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello

```
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r")
content = fileptr.read()
print(content)
```

output:
Hello

Read file through for loop

We can read the file using for loop.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello
Hello world

```
#open the file.txt in read mode. causes an error if no such
file exists.
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r");
#running a for loop
for i in fileptr:
    print(i) # i contains each line of the file
```

output:
Hello

Hello world

Read Lines of the file

readline() method

Python facilitates to read the file line by line by using a function readline() method. The readline() method reads the lines of the file from the beginning, i.e., if we use the readline() method two times, then we can get the first two lines of the file.

Example which contains a function `readline()` that reads the first line of our file "sample.txt" containing three lines.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

```
Hello  
Hello world
```

```
#open the sample.txt in read mode. causes error if no such  
file exists.  
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r");  
#stores all the data of the file into the variable content  
content = fileptr.readline()  
content1 = fileptr.readline()  
#prints the content of the file  
print(content)  
print(content1)  
#closes the opened file  
fileptr.close()
```

output:
Hello

Hello world

We called the `readline()` function two times that's why it read two lines from the file.

readlines() function

Python provides also the `readlines()` method which is used for the reading lines. It returns the list of the lines till the end of file (EOF) is reached.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello world

```
#open the sample.txt in read mode. causes error if no such file exists.
```

```
fileptr = open(r"C:\Users\Username\Desktop\sample.txt ", "r");
```

```
#stores all the data of the file into the variable content
```

```
content = fileptr.readlines()
```

```
#prints the content of the file
```

```
print(content)
```

```
#closes the opened file
```

```
fileptr.close()
```

output:

```
['Hello World']
```

Creating a new file

The new file can be created by using one of the following access modes with the function open().

- x:** it creates a new file with the specified name. It causes an error a file exists with the same name.
- a:** It creates a new file with the specified name if no such file exists. It appends the content to the file if the file already exists with the specified name.
- w:** It creates a new file with the specified name if no such file exists. It overwrites the existing file.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello world

```
#open the file.txt in read mode. causes error if no such file exists.
fileptr = open(r"C:\Users\RR\Desktop\sample.txt", "r")
print(fileptr)
if fileptr:
    print("File opened successfully")
```

output:

```
<_io.TextIOWrapper
name='C:\\Users\\Username\\Desktop\\sample.txt' mode='r'
encoding='UTF-8'>
File opened successfully
```

```
#open the file.txt in append mode. If file not found new file is created
fileptr = open(r"C:\Users\RR\Desktop\sample1.txt", "a")
print(fileptr)
if fileptr:
    print("File opened successfully")
fileptr.close()
```

```
#open the file.txt in write mode. Existing file is overwritten. If file not found
new file is created
fileptr = open(r"C:\Users\RR\Desktop\sample1.txt", "w")
print(fileptr)
if fileptr:
    print("File opened successfully")
fileptr.close()
```

File Pointer positions

Python provides the tell() method which is used to print the byte number at which the file pointer currently exists.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello world

```
# open the file file2.txt in read mode
fileptr = open(r"C:\Users\Username\Desktop\sample.txt", "r")
#initially the filepointer is at 0
```

```
print("The filepointer is at byte :",fileptr.tell())
#reading the content of the file
content = fileptr.read()
#after the read operation file pointer modifies. tell() returns the
#location of the fileptr.
print("After reading, the filepointer is at:",fileptr.tell())
```

output:

*The filepointer is at byte : 0
After reading, the filepointer is at: 11*

Modifying file pointer position

In real-world applications, sometimes we need to change the file pointer location externally since we may need to read or write the content at various locations.

For this purpose, the Python provides us the `seek()` method which enables us to modify the file pointer position externally.

The syntax to use the `seek()` method is given below.

Syntax:

`<file-ptr>.seek(offset[, from])`

offset: It refers to the new position of the file pointer within the file.

from: It indicates the reference position from where the bytes are to be moved. If it is set to 0, the beginning of the file is used as the reference position. If it is set to 1, the current position of the file pointer is used as the reference position. If it is set to 2, the end of the file pointer is used as the reference position.

Create a sample.txt file in "C:\Users\Username\Desktop\sample.txt" and add following lines

Hello world

```
# open the file file2.txt in read mode
fileptr = open(r"C:\Users\Username\Desktop\sample.txt","r")
#initially the filepointer is at 0
print("The filepointer is at byte :",fileptr.tell())

#changing the file pointer location to 10.
fileptr.seek(10)

#tell() returns the location of the fileptr.
print("After reading, the filepointer is at:",fileptr.tell())
```

output:

The filepointer is at byte : 0

After reading, the filepointer is at: 10



smecttechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215